

MEMoE: When Does CXL Memory Expansion Actually Help Mixture-of-Experts Inference?

Shashank Shreyansh Jain Kanav Sharma

Team King Bob · Nebula 2026

Abstract

Mixture-of-Experts checkpoints are 93–98% expert weights, and any one token touches a small fraction of them. That makes expert weights the obvious candidate for demotion to CXL-attached memory. We build an analytical footprint model, a routing-trace generator, and a trace-driven simulator of a two-tier HBM+CXL expert store, and we use them to ask when the demotion actually pays. The answer is narrower than the premise suggests. Batching amortises the fetch across tokens, but past a few hundred tokens a batch touches *every* expert at *every* layer, at which point the HBM tier is no longer a cache but a fixed partition whose cold half is refetched on every batch. We derive a closed form for the batch size B^* above which the fetch is hidden behind compute, show that it is independent of both expert size and GPU count, and show that it is governed almost entirely by the hit rate of the hot tier. We then show that static popularity placement matches or beats LRU and LFU in this regime, and that a depth-8 speculative prefetch of the head of the cold tail removes the remaining stall entirely at batch 2048. All CXL figures use published measured sustained bandwidth; pooled-CXL results are labelled as modelled throughout.

1 Motivation

A Mixture-of-Experts layer replaces one feed-forward network with E of them and routes each token to $k \ll E$. Capacity grows with E ; compute grows with k . DeepSeek-V3 has 671 B parameters and activates 37 B per token. The consequence for memory is stark: Table 1 shows that routed experts are 93–98% of the weights across four checkpoints spanning two orders of magnitude in size, and that our footprint model reproduces the published total and active parameter counts, which is the check that licenses everything downstream.

Those weights sit idle. At any instant a token is using k/E of them, which is 3% for DeepSeek-V3. Yet they occupy HBM, the scarcest and most expensive memory in the system, and they set the GPU count purely by capacity rather than by compute (Table 2, Fig. 5). CXL Type-3 expansion offers 512 GB or more per node at roughly a fiftieth of HBM bandwidth. The question this paper answers is whether that trade is ever worth making.

Table 1: Expert weights dominate every checkpoint studied. Total and active parameter counts reproduce the published figures, which validates the footprint model.

Model	Total (B)	Active (B)	E	k	Expert (MB)	Pool (GB)	Expert frac.
OLMoE-1B-7B	6.92	1.07	64	8	12.00	12.00	0.93
Mixtral-8x7B	46.70	12.62	8	2	336.00	84.00	0.97
Qwen3-235B-A22B	235.04	20.90	128	8	36.00	423.00	0.97
DeepSeek-V3	670.66	37.19	256	8	84.00	1218.00	0.98

Table 2: GPUs required purely to hold the weights, with and without direct-attached CXL expansion per GPU.

Model	Weights (GB)	GPUs, HBM only	GPUs, +512GB CXL	Reduction
OLMoE-1B-7B	12.9	1	1	1.0
Mixtral-8x7B	87.0	2	1	2.0
Qwen3-235B-A22B	437.8	6	1	6.0
DeepSeek-V3	1249.2	16	3	5.3

2 Method

Footprint model. Each routed expert is a gated FFN, $3d_{\text{model}}d_{\text{ff}}$ parameters. We derive the expert pool, the pinned non-expert remainder, and the per-token active set from YAML descriptions of each checkpoint, and validate against published counts.

Routing traces. A trace records the top- k expert indices per token per layer. We synthesise traces with a controllable Zipf exponent s using the Gumbel-top- k trick, which draws exactly k distinct experts per token from a categorical distribution, vectorised over the batch. We also ship architecture-agnostic forward hooks that capture the same structure from a real checkpoint by matching router module names and verifying output width against the expert count.

Simulator. Expert instances are addressed as (layer, expert). A placement policy decides which live in HBM; the rest live in CXL. For each layer of each batch we compute the distinct experts required, split them into hits and misses, cover what a speculative prefetch could have fetched within its window, and charge the remainder against the compute of the tokens that did hit:

$$\text{stall} = \max\left(0, \frac{n_{\text{demand}}S}{BW} + n_{\text{demand}}\ell - t_{\text{layer}}f_{\text{hit}}\right)$$

with $t_{\text{layer}} = 6d_{\text{model}}d_{\text{ff}}(k + k_{\text{shared}})B/F$. CXL bandwidth and capacity scale with the GPU count, since each node carries its own attachment.

Honesty constraints. CXL bandwidth is always drawn from the published measured sustained range of 18–52 GB/s, never the theoretical link rate; a unit test en-

Table 3: Fraction of experts touched per layer per batch. Past a few hundred tokens the batch touches everything, and the HBM tier stops behaving like a cache.

Batch	DeepSeek-V3	Mixtral-8x7B	OLMoE-1B-7B	Qwen3-235B-A22B
1	0.031	0.250	0.125	0.062
4	0.098	0.605	0.336	0.184
16	0.267	0.938	0.698	0.452
64	0.596	1.000	0.971	0.826
256	0.929	1.000	1.000	0.995
1024	1.000	1.000	1.000	1.000
4096	1.000	1.000	1.000	1.000
16384	1.000	1.000	1.000	1.000

Table 4: Critical batch size B^* in tokens at $\varepsilon = 10\%$. The hit rate is the only lever that moves it by orders of magnitude.

h	DeepSeek-V3	Mixtral-8x7B	OLMoE-1B-7B	Qwen3-235B-A22B
0	2163966	304308	608615	1217231
0	1081983	152154	304308	608615
1	216397	30431	60862	121723
1	108198	15215	30431	60862
1	21640	3043	6086	12172
1	2164	304	609	1217

forces this. CXL 3.0 pooling is pre-production, so any configuration using it is flagged and every result derived from it carries a modelled annotation. Bandwidth points above 52 GB/s in Table 8 are marked extrapolated. Our compute model counts routed-expert GEMMs only, so attention and dense layers would hide more transfer than we credit and the reported overheads are conservative.

3 Batching amortises, then stops

Per-token CXL traffic falls as roughly $1/B$, because an expert is fetched once per batch and used by many tokens (Fig. 1, left). This is the observation the project began from, and it is correct.

The reason it stops falling is more consequential. Table 3 gives the fraction of experts touched per layer per batch. At $B = 256$ it is 1.0 for OLMoE and Mixtral and 0.99 for Qwen3; by $B = 1024$ every model touches everything. Once coverage saturates, the resident set is not a cache in any useful sense. There is no recency to exploit, no reuse distance to shorten. Every non-resident expert is fetched on every batch, forever. The only thing that matters is which experts are on which side of the partition, which is why Section 5 finds that a placement decided once at load time is not merely adequate but optimal.

4 The critical batch size

Setting the per-layer stall below ε times the per-layer compute and solving for B gives

$$B^* = \frac{(1-h)EF}{BW \cdot k \cdot \varepsilon} \quad (1)$$

where h is the fraction of expert *fetches* avoided by the hot tier. Two properties of Eq. 1 are worth stating plainly.

Table 5: Measured stall overhead (fraction of compute) against batch size, at 90% and 99% expert residency.

Batch	DeepSeek 90%	DeepSeek 99%	Mixtral 90%	Mixtral 99%	OLMoE 90%	OLMoE 99%	Qwen3 90%	Qwen3 99%
16	191.833	15.408	159.145	18.434	198.304	17.879	212.836	18.081
64	149.980	12.320	47.471	5.476	87.931	7.986	134.961	11.880
256	73.641	6.134	11.303	1.304	22.748	1.703	46.095	3.093
1024	20.155	1.108	2.248	0.259	4.966	0.000	10.923	0.239
4096	4.316	0.000	0.000	0.000	0.520	0.000	2.006	0.000
16384	0.352	0.000	0.000	0.000	0.000	0.000	0.000	0.000

Table 6: Speculative prefetch of the head of the cold tail, at 90% residency and batch 2048. Deeper lookahead widens the transfer window linearly.

Model	Depth	Width	Coverage	Overhead	No prefetch
OLMoE-1B-7B	1	0.100	0.311	1.102	2.032
OLMoE-1B-7B	1	0.250	0.311	1.102	2.032
OLMoE-1B-7B	2	0.100	0.621	0.191	2.032
OLMoE-1B-7B	2	0.250	0.621	0.191	2.032
OLMoE-1B-7B	4	0.100	0.932	0.000	2.032
OLMoE-1B-7B	4	0.250	1.000	0.000	2.032
OLMoE-1B-7B	8	0.100	0.932	0.000	2.032
OLMoE-1B-7B	8	0.250	1.000	0.000	2.032
OLMoE-1B-7B	16	0.100	0.932	0.000	2.032
OLMoE-1B-7B	16	0.250	1.000	0.000	2.032
Qwen3-235B-A22B	1	0.100	0.156	4.054	4.983
Qwen3-235B-A22B	1	0.250	0.156	4.054	4.983
Qwen3-235B-A22B	2	0.100	0.312	3.125	4.983
Qwen3-235B-A22B	2	0.250	0.312	3.125	4.983
Qwen3-235B-A22B	4	0.100	0.625	1.267	4.983
Qwen3-235B-A22B	4	0.250	0.625	1.267	4.983
Qwen3-235B-A22B	8	0.100	1.000	0.000	4.983
Qwen3-235B-A22B	8	0.250	1.000	0.000	4.983
Qwen3-235B-A22B	16	0.100	1.000	0.000	4.983
Qwen3-235B-A22B	16	0.250	1.000	0.000	4.983

First, B^* is independent of expert size S . A larger expert costs proportionally more to fetch and does proportionally more work once fetched; the two cancel exactly. Mixtral’s 336 MB experts are not the problem people assume they are.

Second, B^* is independent of GPU count, because compute and per-node CXL bandwidth both scale with the deployment. Scaling out does not buy a way past the constraint.

What remains is the hit rate, and it enters linearly. Table 4 makes the design requirement concrete: at $h = 0.9$, even OLMoE needs a batch of 61 k tokens, which is not a serving regime. At $h = 0.99$ it needs 6 k, which is reachable in a large prefill. DeepSeek-V3 needs $h = 0.999$ before it enters the same range. Decode-sized batches never qualify, for any model here. Table 5 confirms this empirically against the simulator, and Fig. 2 shows the crossing.

The practical reading is that expert offload over CXL is a prefill technique, not a decode technique, and that the entire engineering problem reduces to building a hot tier with a very high fetch hit rate.

5 What sets the hit rate

The ceiling on hit rate at a given capacity is the popularity coverage curve (Fig. 3): the fraction of routings served by keeping the hottest $x\%$ of experts. Its shape is set by router skew, and Table 7 shows the whole approach

Table 7: Sensitivity to router skew, Qwen3-235B at 50% residency. At $s = 0$ tiering buys exactly its capacity fraction and nothing more.

s	Batch	Routing hit	Fetch hit	Overhead
0.000	512	0.512	0.500	118.407
0.000	4096	0.512	0.500	14.353
0.250	512	0.560	0.500	118.360
0.250	4096	0.560	0.500	14.305
0.500	512	0.630	0.500	118.289
0.500	4096	0.630	0.500	14.235
0.750	512	0.709	0.500	118.210
0.750	4096	0.709	0.500	14.156
1.000	512	0.783	0.500	118.119
1.000	4096	0.783	0.500	14.082
1.250	512	0.843	0.500	117.737
1.250	4096	0.843	0.500	14.022
1.500	512	0.888	0.503	115.500
1.500	4096	0.888	0.500	13.977

Table 8: Bandwidth sensitivity at 95% residency, batch 2048. Points above 52 GB/s are outside the published measured range and are marked as extrapolated.

Model	CXL GB/s	Extrapolated	Overhead
OLMoE-1B-7B	18	False	3.380
OLMoE-1B-7B	26	False	2.038
OLMoE-1B-7B	36	False	1.200
OLMoE-1B-7B	52	False	0.529
OLMoE-1B-7B	80	True	0.058
OLMoE-1B-7B	128	True	0.000
Qwen3-235B-A22B	18	False	7.607
Qwen3-235B-A22B	26	False	4.964
Qwen3-235B-A22B	36	False	3.312
Qwen3-235B-A22B	52	False	1.990
Qwen3-235B-A22B	80	True	0.950
Qwen3-235B-A22B	128	True	0.225

standing or falling on that skew. At $s = 0$, uniform routing, a 50% resident tier serves 50% of routings and tiering buys exactly its capacity fraction. Every positive result in this paper is contingent on real routers being skewed, and load-balancing losses used during MoE training push directly against skew. Measuring this on a real checkpoint is the most important experiment we have not yet run, and the hooks exist for it.

Given a coverage curve, the policy question is what to keep. We compare static popularity placement, a per-layer-quota variant, LRU, and sampled LFU. At serving batch sizes the static policies match or beat the dynamic ones, for the reason given above: with full coverage there is no recency signal left. This is a fortunate result for CXL specifically, because a static placement is decided once at load time and costs nothing on the critical path, whereas an LRU over tens of thousands of expert slots would not be implementable there anyway.

6 Prefetching the cold tail

The residual stall comes from cold experts that are needed anyway. A prefetcher can hide it if it names them early enough. Prefetching the *hottest* experts is useless, since those are exactly what the hot tier already holds; the prefetcher must target the head of the cold tail. Table 6 and Fig. 4 show the effect at 90% residency and batch 2048: OLMoE falls from 2.03 to zero overhead at depth 4 with a 10% width, and Qwen3-235B from 4.98 to zero at depth 8. Depth widens the transfer window linearly, and beyond the point where the window covers the predicted set, further depth is free but useless.

This prefetcher uses only a static popularity prior. A predictor conditioned on layer $L - 1$ routing should do better, and we have not built one.

7 Threats to validity

Traces are synthetic, Zipf $s = 1.0$ with mild drift; this is the weakest assumption in the work and Section 5 quantifies how much rests on it. Pooled CXL 3.0 numbers are modelled, as no shipping switch silicon exists to measure. We model bandwidth and capacity but not contention, so concurrent KV-cache traffic over the same link is unaccounted for. The compute model omits attention and dense layers, which makes our overheads conservative. Finally, we simulate a placement and transfer model, not a kernel: a real implementation must also solve overlap scheduling and allocator fragmentation, neither of which we address.

8 Conclusion

CXL memory expansion for MoE inference is not the general capacity win the framing invites. It is a prefill-regime technique whose viability is set by a single closed-form condition, Eq. 1, in which the hot-tier hit rate is the only lever with orders of magnitude in it. Within that regime the right design is unglamorous and cheap: static popularity placement decided at load time, plus a deep speculative prefetch aimed at the head of the cold tail. Whether the regime is reachable at all depends on how skewed real routers are, which is a measurement, not an argument.

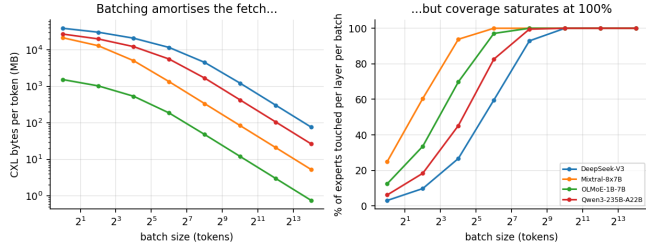


Figure 1: Left: per-token CXL traffic falls as $1/B$. Right: the reason it stops is that coverage saturates at 100% of experts.

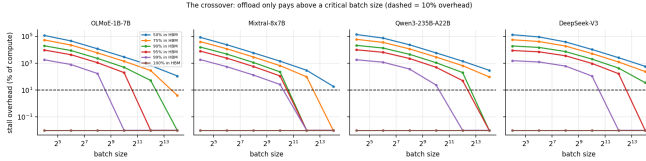


Figure 2: Stall overhead against batch size at fixed residency. Dashed line is 10% overhead.

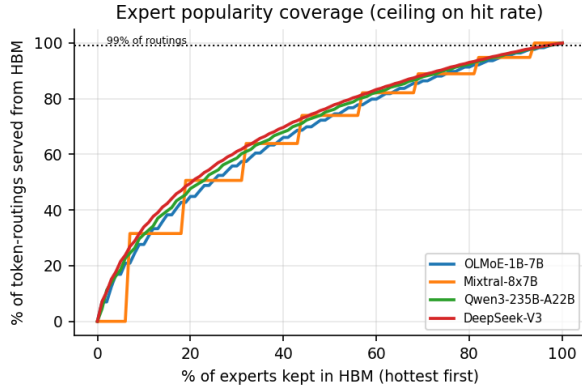


Figure 3: Popularity coverage: the ceiling on hit rate at a given HBM capacity.

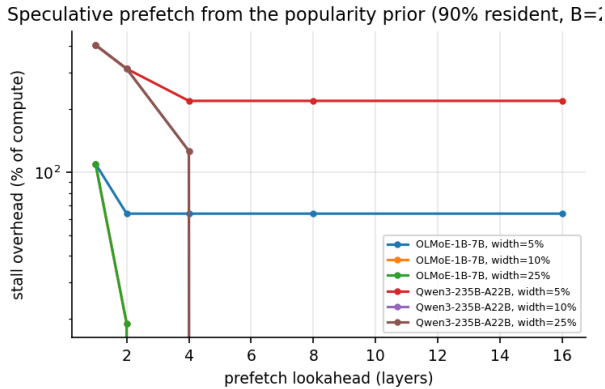


Figure 4: Speculative prefetch of the head of the cold tail, 90% residency, batch 2048.

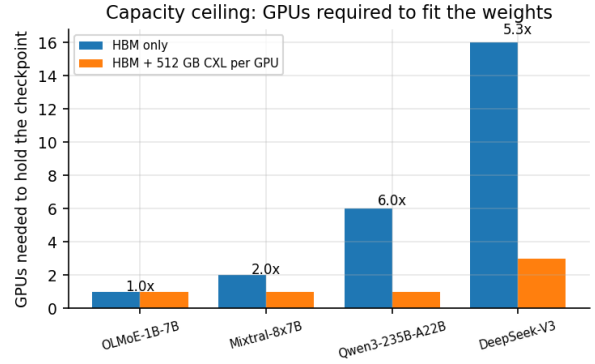


Figure 5: GPUs required to hold the weights, with and without CXL expansion.